HOME BUDGET TRACKER

HOME BUDGET AND EXPENSE TRACKING SYSTEM

A project report submitted in partial fulfillment of the requirements for the Bachelor of Technology in Computer Science and Engineering

PROJECT TEAM DETAILS

S. No.	Team Member	Role / Contribution
1	Peetha Geethika	Database – MySQL Workbench
2	Zaddu Lakshmi Sai Charitha	Frontend Website Design – HTML, CSS, JavaScript, Bootstrap (optional)
3	Eniya J	Data Analysis and Visualization – Pandas, NumPy, Plotly
4	Thoka Brahmeswari	Budget Planner, Integration, Testing and Documentation
5	Boya Santhosh	Backend Development – Python, Flask

UNDER THE GUIDANCE OF
Mr. Mohammed Valli

Department of Computer Science and Engineering
Mohan Babu University
Academic Year: 2026–2027

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Existing System
6. Proposed System
7. Software and Hardware Requirements
8. Technologies Used
9. System Architecture
10. Database Design
11. Module Description
12. System Implementation
13. Results and Screenshots
14. Testing
15. Conclusion
16. Future Enhancements
17. References

1. ABSTRACT

Home Budget Tracker is a web-based application developed to help users record, organize, monitor, and analyze their personal or household income and expenses. It provides a centralized platform for maintaining financial transactions and reviewing the overall financial position through a dashboard.

The application supports transaction management, including adding, viewing, editing, and deleting income and expense records. The dashboard presents key financial information such as total income, total expenses, available balance, and savings rate. The project also includes budgeting functionality for monthly and category-wise planning, month comparison, financial analysis, and savings-goal management.

The system is developed using Python and the Flask web framework for backend processing, HTML, CSS, and JavaScript for the user interface, and a MySQL-compatible relational database for persistent storage. Pandas and NumPy support data processing and calculations, while Plotly is used for graphical visualization where configured. The application is deployed through Render and connects to a hosted database using protected configuration settings.

Home Budget Tracker demonstrates the practical integration of web development, database management, budgeting logic, and data visualization to make everyday financial record keeping more organized and understandable.

2. INTRODUCTION

Managing personal and household finances requires regular tracking of income, expenses, savings, and planned budgets. When financial details are recorded manually in notebooks or separate spreadsheets, users may find it difficult to maintain consistent records, calculate totals, identify spending patterns, and compare actual expenses with planned limits.

Home Budget Tracker is designed to address these difficulties through a web-based application. Users can enter financial transactions, maintain records, view dashboard summaries, analyze spending, compare monthly activity, manage budgets, and track savings goals.

The project combines a browser-based interface with a Flask backend and a relational database. The frontend collects user input and displays results. Flask processes requests, applies application logic, and communicates with the database. Stored transaction data is then used to calculate summaries and prepare analytical visualizations.

2.1 Project Overview

The application provides a dashboard with financial summaries and navigation to the Transactions, Analysis, Monthly Budget, Compare Months, Category Budget Manager, and savings-goal functionality. Users can manage their financial records and review the information in a structured manner.

2.2 Purpose

The purpose of the project is to provide a convenient digital method for maintaining income and expense records, planning monthly spending, and understanding financial activity using summaries and charts.

3. PROBLEM STATEMENT

Manual tracking of household income and expenses can become time-consuming and error-prone as the number of transactions increases. Records may be scattered across notebooks or spreadsheets, making it difficult to retrieve information, calculate monthly totals, compare spending categories, and monitor savings.

Without a centralized system, users may not have a clear view of their available balance or whether their spending is within planned budget limits. Preparing reports and identifying spending trends may also require repeated manual calculations.

Therefore, a web-based system is required to store financial transactions in an organized database, perform relevant calculations, support budget planning, and present useful summaries and visualizations. Home Budget Tracker addresses these needs through a single integrated application.

4. OBJECTIVES

- To develop a user-friendly web application for personal and household budget management.
- To provide facilities for recording income and expense transactions.
- To allow users to view, edit, and delete stored transaction records.
- To store financial information in a structured relational database.
- To display total income, total expenses, available balance, and savings rate on the dashboard.
- To provide monthly budget and category-wise budget management.
- To compare financial activity across selected months.
- To analyze transaction information and present useful charts.
- To support savings-goal creation and progress tracking.
- To deploy the application so that it can be accessed through a web browser.

5. EXISTING SYSTEM

In a traditional approach, users maintain household financial records using paper registers, calculators, or spreadsheet applications. Income and expense details are entered manually, and totals are calculated periodically.

5.1 Limitations of the Existing System

- Maintaining records manually requires time and regular effort.
- Separate records may lead to inconsistent or duplicated information.
- Calculating monthly income, expenses, and balance can require manual work.
- Identifying high-spending categories may be difficult.
- Comparing actual expenses with budget limits is not always convenient.
- Tracking progress toward savings goals may require separate calculations.
- Preparing charts and reports from raw records can be time-consuming.

6. PROPOSED SYSTEM

The proposed Home Budget Tracker is a database-backed web application that centralizes financial information and provides features for transaction maintenance, budget planning, analysis, and savings tracking. The application processes user requests through Flask and stores records in a MySQL-compatible database.

The dashboard displays important financial summaries. The transaction section supports the maintenance of income and expense records. Monthly and category budget sections allow users to plan spending limits. The month-comparison and analysis sections help users review financial activity. Savings goals allow users to record a target amount, saved amount, and target date.

6.1 Advantages of the Proposed System

- Centralized storage of financial transactions.
- Convenient addition, editing, viewing, and deletion of records.
- Automatic calculation of financial summaries.
- Monthly and category-wise budget management.
- Comparison of financial activity across months.
- Visual representation of income and expense information.
- Tracking of savings goals and progress.
- Browser-based access through the deployed application.

7. SOFTWARE AND HARDWARE REQUIREMENTS

7.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent and above.
- RAM: Minimum 4 GB recommended.
- Storage: Sufficient free storage for the source code, Python environment, and development tools.
- Display: Standard laptop or desktop display.
- Input devices: Keyboard and mouse.
- Internet connection: Required for hosted database access and deployed application use.

7.2 Software Requirements

- Operating System: Windows or another operating system supporting Python.
- Programming Language: Python.
- Backend Framework: Flask.
- Frontend: HTML, CSS, and JavaScript.
- Database: MySQL-compatible relational database.
- Database Tool: MySQL Workbench.
- Development Environment: Visual Studio Code.
- Data Processing: Pandas and NumPy.
- Data Visualization: Plotly.
- Deployment Platform: Render.
- Web Browser: Google Chrome or another modern browser.

7.3 Functional Requirements

- The application should accept income and expense details.
- Users should be able to view and maintain transaction records.
- The dashboard should calculate and display financial summaries.
- The application should support monthly and category budget features.
- Users should be able to compare selected months.
- The analysis section should process transaction data and display supported visualizations.
- Users should be able to maintain savings goals and review their progress.

7.4 Non-Functional Requirements

- Usability: The interface should be understandable and easy to navigate.
- Reliability: Database operations should store and retrieve records consistently.
- Security: Database credentials and secret configuration values should be kept outside public source code.
- Maintainability: Backend logic, templates, and database operations should be organized.
- Performance: Common operations should complete within a reasonable time for normal personal-use data.
- Availability: The deployed application should be reachable when the hosting service and database are available.

8. TECHNOLOGIES USED

8.1 Python

Python is the primary programming language used for backend application logic, database operations, calculations, and data processing.

8.2 Flask

Flask is a lightweight Python web framework used to define routes, process browser requests, render templates, and connect the frontend with backend functionality.

8.3 HTML

HTML provides the structure of the application pages, including forms, navigation, tables, and dashboard sections.

8.4 CSS

CSS is used to style the pages, layouts, colors, spacing, cards, forms, and responsive interface elements.

8.5 JavaScript

JavaScript provides client-side interactivity where required by the user interface.

8.6 MySQL

MySQL is used as the relational database technology for storing transaction, budget, and savings-goal information.

8.7 MySQL Connector

MySQL Connector/Python allows the Flask application to communicate with the database and execute SQL operations.

8.8 Pandas

Pandas supports the organization, grouping, and analysis of transaction data.

8.9 NumPy

NumPy supports numerical operations and calculations used by the data-processing component.

8.10 Plotly

Plotly is used to prepare interactive charts and graphical representations of financial information.

8.11 Render

Render is used to host and deploy the Flask web application. Deployment configuration connects the application to the hosted database.

9. SYSTEM ARCHITECTURE

Home Budget Tracker follows a three-layer web application architecture consisting of the presentation layer, application layer, and data layer. The analysis component uses transaction information retrieved from the database.

9.1 Architecture Components

1. Presentation Layer (Frontend)

HTML, CSS, and JavaScript provide the browser interface. Users access the dashboard, transactions, analysis, budget management, month comparison, and savings-goal features.

2. Application Layer (Backend)

Python and Flask receive user requests, process form data, perform calculations, manage application routes, and communicate with the database.

3. Data Layer

The MySQL-compatible database stores transaction details and budget-related information for persistent access.

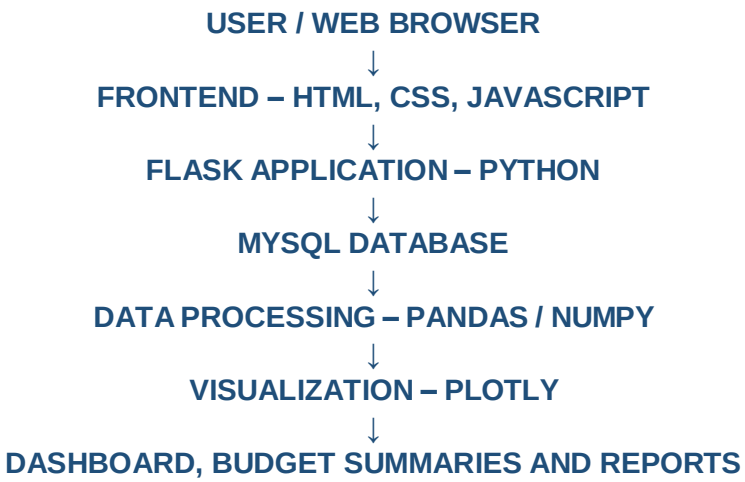
4. Analysis and Visualization Layer

Pandas and NumPy support data processing and calculations. Plotly presents supported results through charts.

9.2 Working Flow

1. The user opens Home Budget Tracker in a web browser.
2. The frontend displays the selected page and its forms or summaries.
3. The user submits a transaction, budget, or savings-goal request.
4. The Flask backend receives and validates the submitted information.
5. The backend performs the required database operation.
6. The database stores the new information or returns existing records.
7. The backend calculates summaries or prepares data for analysis.
8. The application displays the result, table, dashboard summary, or chart to the user.

9.3 Architecture Flow Diagram



10. DATABASE DESIGN

The application uses a relational database to maintain financial information. The database design supports transaction records, monthly and category budget information, and savings goals. The exact production schema should be checked against the database used by the deployed application before final submission.

10.1 Transactions Table

The Transactions table stores income and expense entries. The development workflow confirms fields including transaction identification, user association, transaction type, description, amount, date, category, budget month, and creation timestamp.

Field	Purpose
id	Unique identifier for a transaction; primary key.
user_id	Identifies the user associated with the record, where user association is enabled.
type	Specifies whether the transaction is income or expense.
description	Stores a short description of the transaction.
amount	Stores the transaction amount.
transaction_date	Stores the date of the transaction.
category	Stores the transaction category.
budget_month	Stores the month associated with budget tracking.
created_at	Stores the record creation timestamp.

10.2Monthly Budget Table

The monthly budget functionality stores planned budget information for a selected month. The project workflow includes monthly budget settings. Confirm the exact table name and column names from the final database before submission.

10.3Category Budget Table

The Category Budget Manager supports category-wise spending limits. It is used to maintain planned amounts for individual expense categories and compare them with recorded spending. Confirm the exact production table name and attributes in the database.

10.4Savings Goals Table

The savings-goal functionality stores a target and tracks the amount saved toward that target. The project code discussed during development uses the following fields:

Field	Purpose
id	Unique identifier for a savings goal.
user_id	Associates the goal with a user.
goal_name	Name or description of the savings goal.

target_amount	Total amount planned for the goal.
saved_amount	Amount saved so far.
target_date	Target date for achieving the goal.
created_at	Date and time when the goal was created.

10.5Database Relationships and Integrity

Where user accounts are enabled, transaction and savings-goal records are associated with the relevant user through user_id. Primary keys identify individual records. Foreign-key constraints should be documented only where they are present in the actual database schema.

Important schema note: confirm the exact database name, table names, data types, primary keys, foreign keys, and constraints in MySQL Workbench or the hosted database console. Do not treat this descriptive report as a substitute for the live schema.

11. MODULE DESCRIPTION

11.1 Dashboard Module

The Dashboard module provides an overview of financial activity. It displays total income, total expenses, available balance, and savings rate. These values are calculated from the stored transaction information.

11.2 Transaction Management Module

This module allows users to add, view, edit, and delete financial transactions. Each record contains information such as transaction type, description, amount, date, and category.

11.3 Transaction Listing Module

The transaction listing page displays saved financial records in an organized format so that users can review entries and use the available edit or delete actions.

11.4 Analysis Module

The Analysis module processes transaction data and presents supported financial summaries and charts. Pandas and NumPy can be used for grouping and calculations, while Plotly provides graphical visualizations.

11.5 Monthly Budget Module

This module allows users to enter or review a planned budget for a month and compare the planned amount with recorded expenses.

11.6 Compare Months Module

The Compare Months module allows users to review financial activity across selected months. It helps identify changes in income or expenses over time.

11.7 Category Budget Manager

This module supports budget planning for individual categories. Users can maintain category limits and review spending in relation to the planned amounts.

11.8 Savings Goals Module

The Savings Goals module allows users to define a goal name, target amount, saved amount, and target date. It provides a way to monitor progress toward a planned savings target.

11.9 User Access and Session Module

The application includes login and logout navigation. Session and access-control behavior should be described in accordance with the authentication implementation present in the final source code.

12. SYSTEM IMPLEMENTATION

The application is implemented using Python and Flask for backend processing, HTML/CSS/JavaScript for the browser interface, and a MySQL-compatible database for persistent storage. The implementation combines transaction operations, budget calculations, savings-goal management, and analytical presentation.

12.1 Frontend Implementation

The frontend is organized into pages for the dashboard, transactions, analysis, monthly budget settings, month comparison, category budget management, and savings goals. HTML defines page structure and forms, CSS controls the visual presentation, and JavaScript is used where client-side interaction is required.

12.2 Backend Implementation

Flask routes receive browser requests and perform the required operations. The backend handles form submissions, retrieves transaction records, updates or deletes selected records, and prepares data for dashboard summaries and analysis pages.

12.3 Database Connectivity

The application uses Python database connectivity code to communicate with the MySQL-compatible database. SQL statements are used to insert, retrieve, update, and delete records. Connection configuration and credentials should be supplied through environment variables or protected deployment settings rather than being published in the source code.

12.4Financial Calculations

The dashboard calculates total income and total expenses from the available transaction records. Available balance is calculated as income minus expenses. Savings rate is calculated as the available balance divided by total income and expressed as a percentage when total income is greater than zero.

12.5Budget and Savings Implementation

Monthly and category-wise budget features compare planned amounts with recorded expense values. The savings-goal component maintains the target amount and current saved amount so that progress can be reviewed. The precise calculation and warning rules should match the final application code.

12.6Data Analysis and Visualization

Transaction information is retrieved from the database and processed to prepare summaries. Pandas and NumPy support data manipulation and numerical calculations. Plotly is used for visual presentation where charts are configured in the application.

12.7Deployment

The Flask application is deployed using Render. The deployed service requires the correct start command, installed Python dependencies, environment configuration, and network access to the hosted database. Where the database requires TLS, the required CA certificate must be configured securely as a deployment secret file or through the hosting provider's supported certificate mechanism. Database passwords, private keys, and secret values must not be included in this report.

12.8Live Application Access

The deployed Home Budget Tracker application can be accessed through the

following URL: https://homebudget-tracker.onrender.com

12.9Running the Application Locally

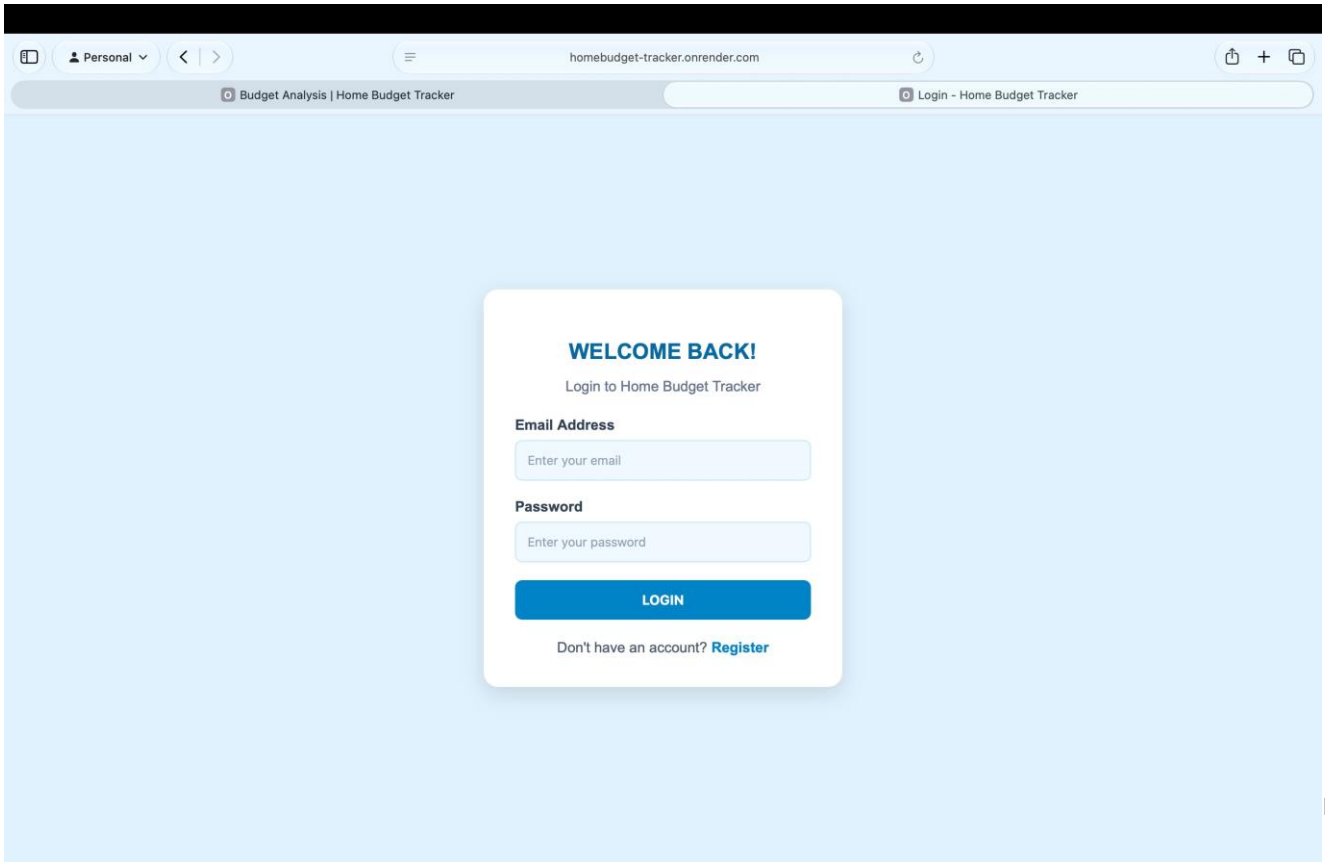
9. Install Python and the required development tools.
10. Obtain the project source code and open the project folder in Visual Studio Code.
11. Install the dependencies listed in requirements.txt.
12. Create or configure the required database and tables.
13. Set the database connection values and secret key through local environment configuration.
14. Start the Flask application using the project's configured run command.
15. Open the local application address in a web browser and test the available modules.

13. RESULTS AND SCREENSHOT

The completed Home Budget Tracker provides a web interface for maintaining financial transactions and reviewing budget-related information. The following screenshots should be inserted from the actual running application to demonstrate each implemented feature.

13.1Login Page

Shows the login interface used to access the application, if authentication is enabled.



n.

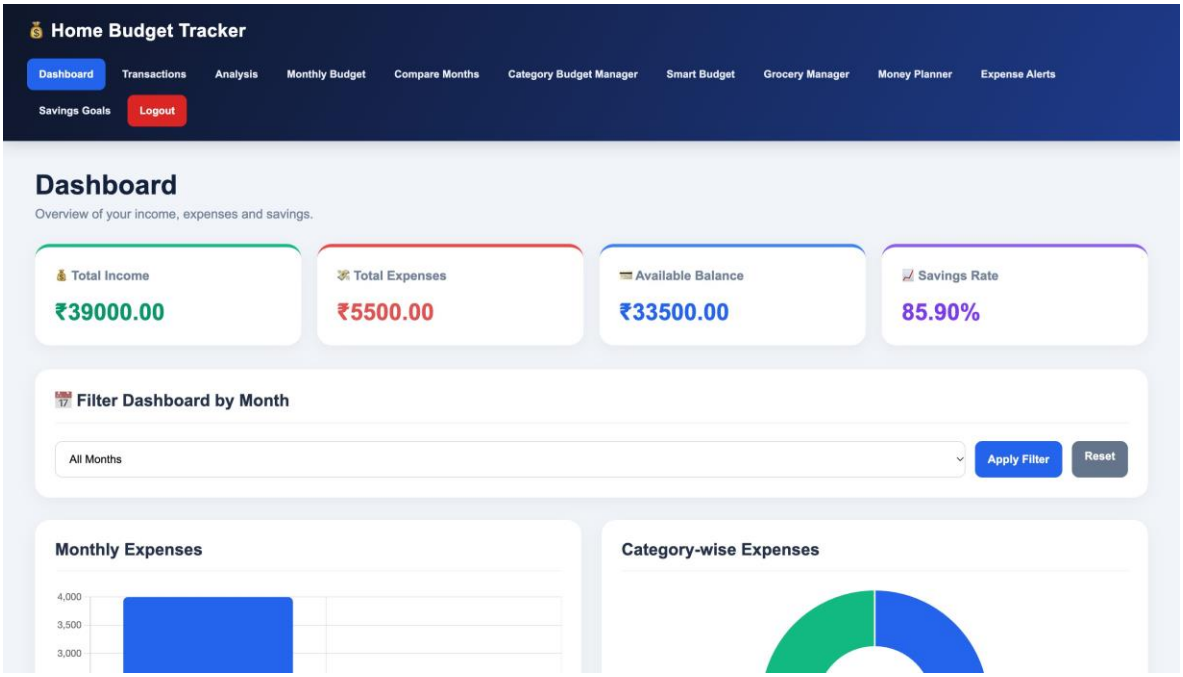


Fig. 13.2: Dashboard Page

13.3Transactions Page

Shows the transaction form and the list of saved income and expense records.

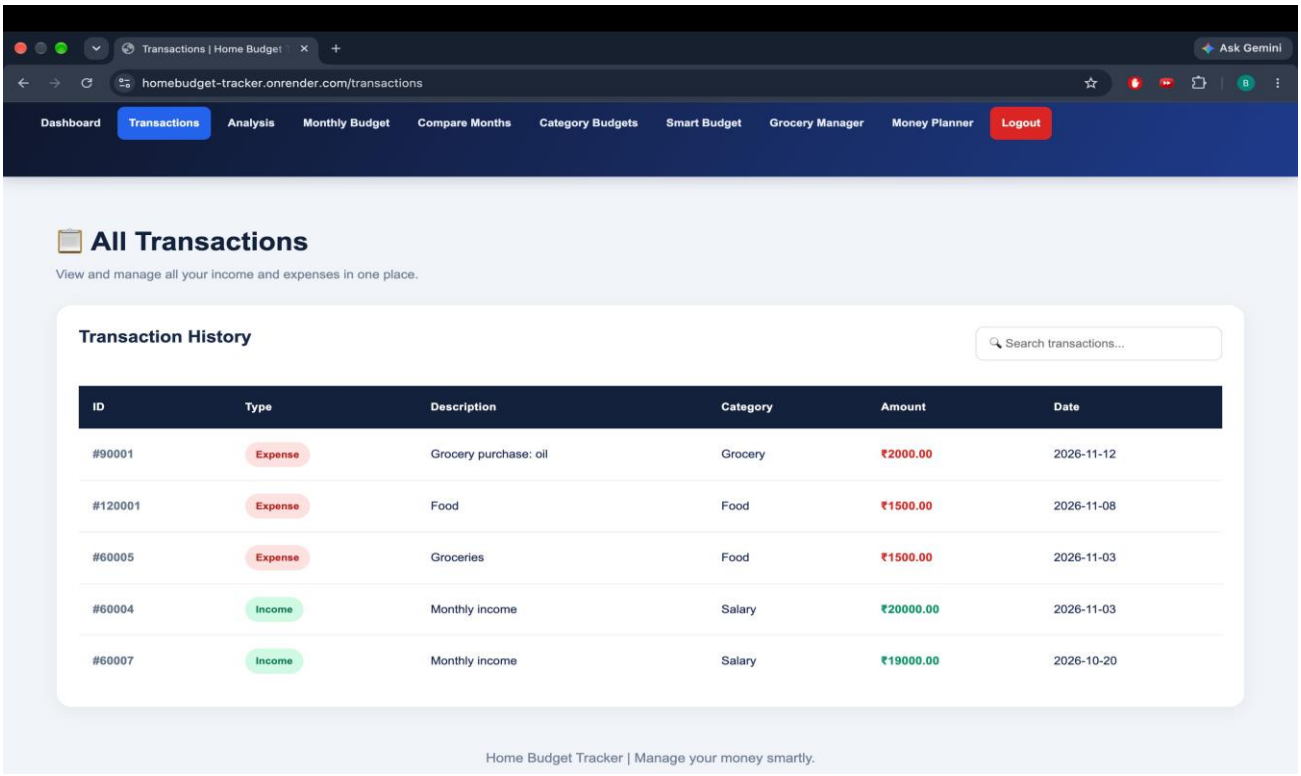


Fig. 13.3: Transactions Page

13.4 Analysis Page

Shows charts or summaries generated from the available transaction data.

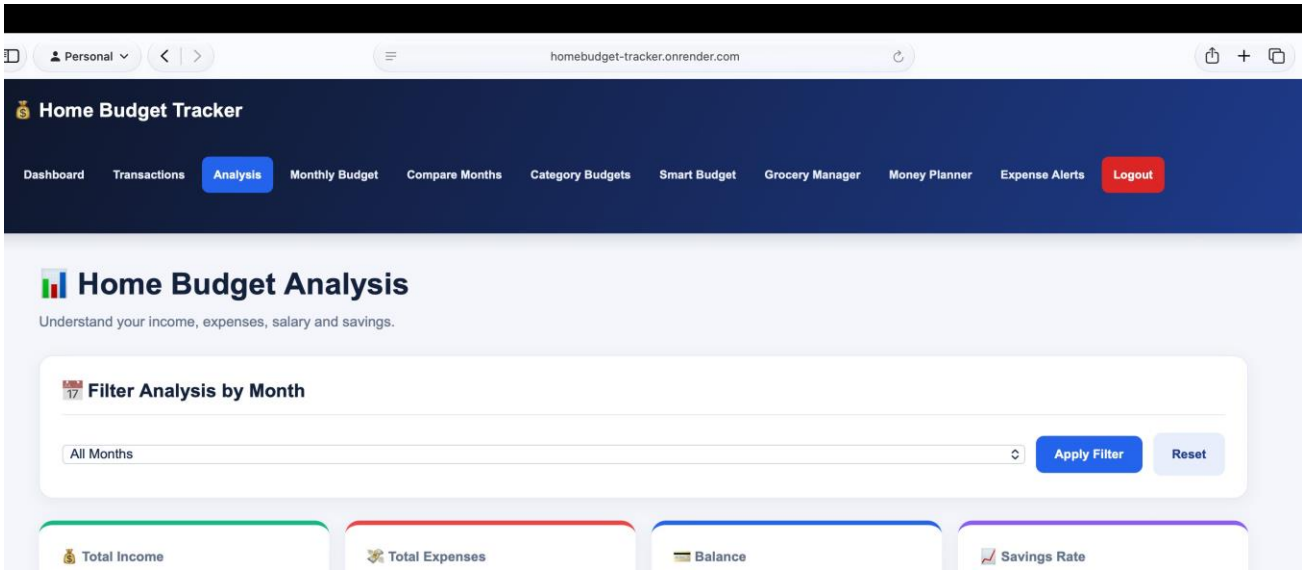


Fig. 13.4: Analysis Page

13.5 Monthly Budget Page

Shows the monthly budget form and the corresponding budget summary.

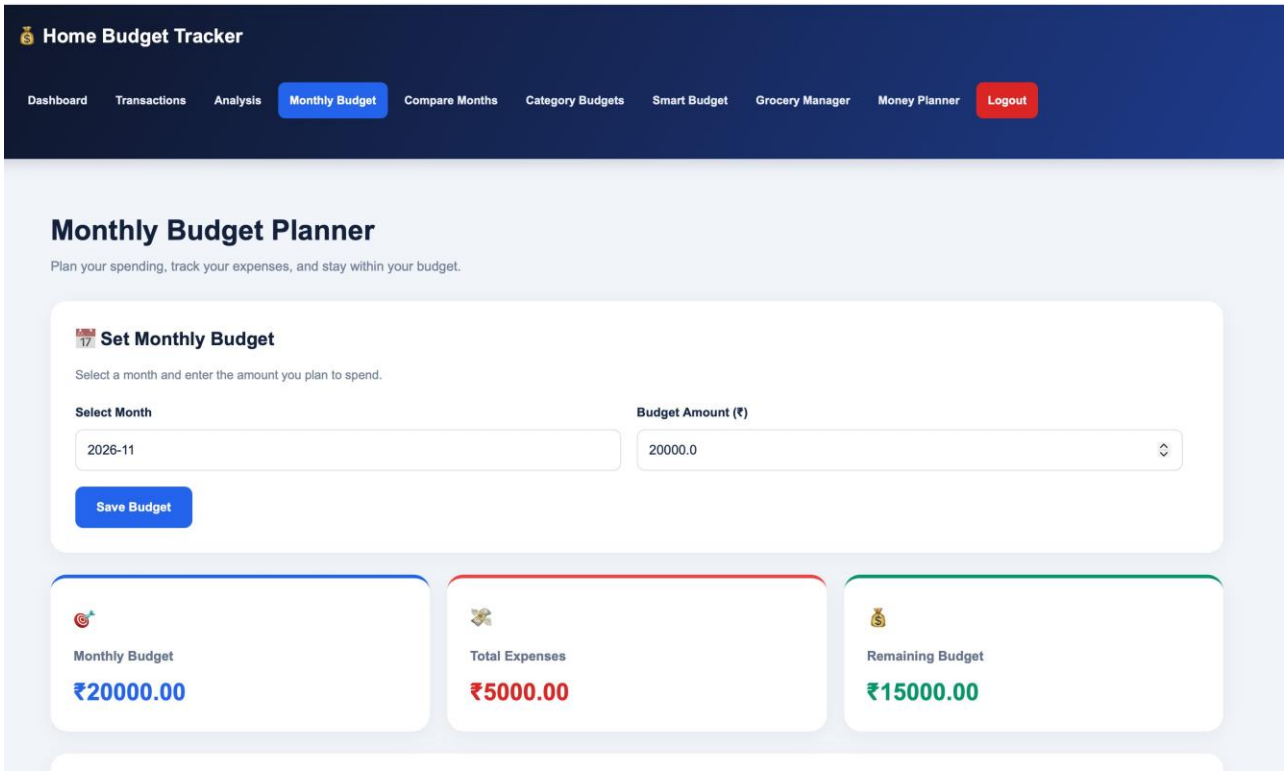


Fig. 13.5: Monthly Budget Page

13.6 Compare Months Page

Shows the comparison of financial activity between selected months.

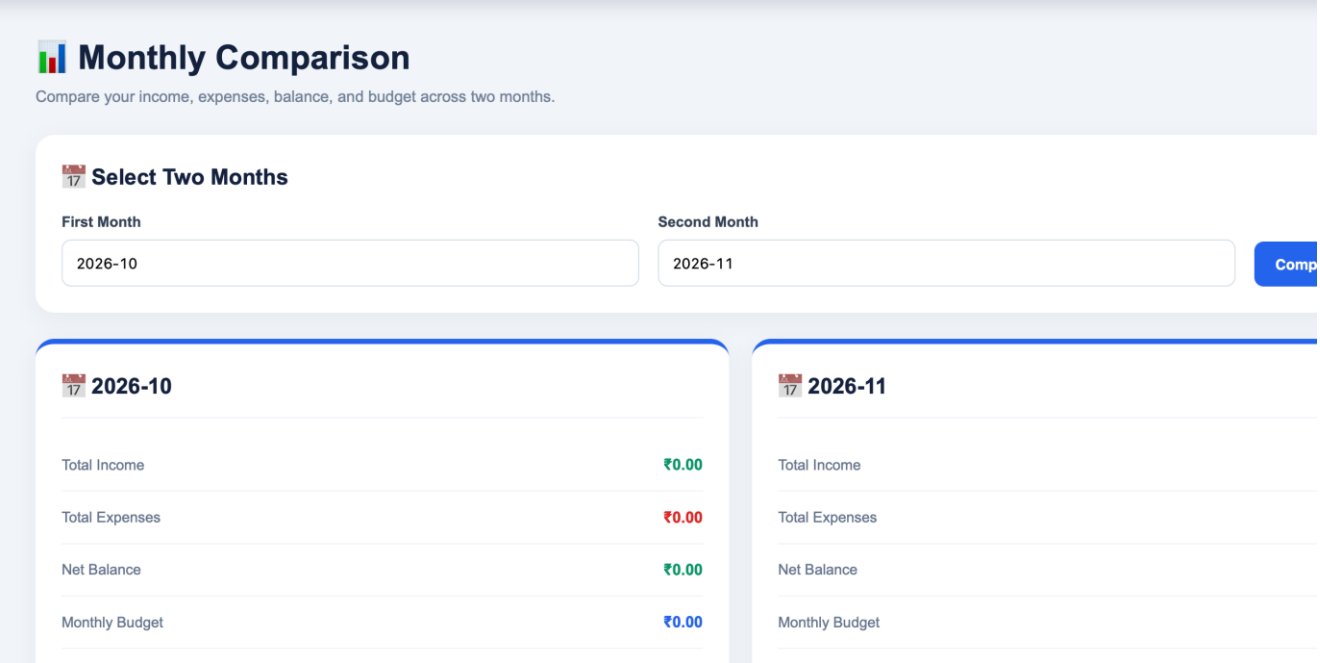


Fig. 13.6 Compare Months Page

13.7 Category Budget Manager

Shows category-wise budget settings and related spending information.

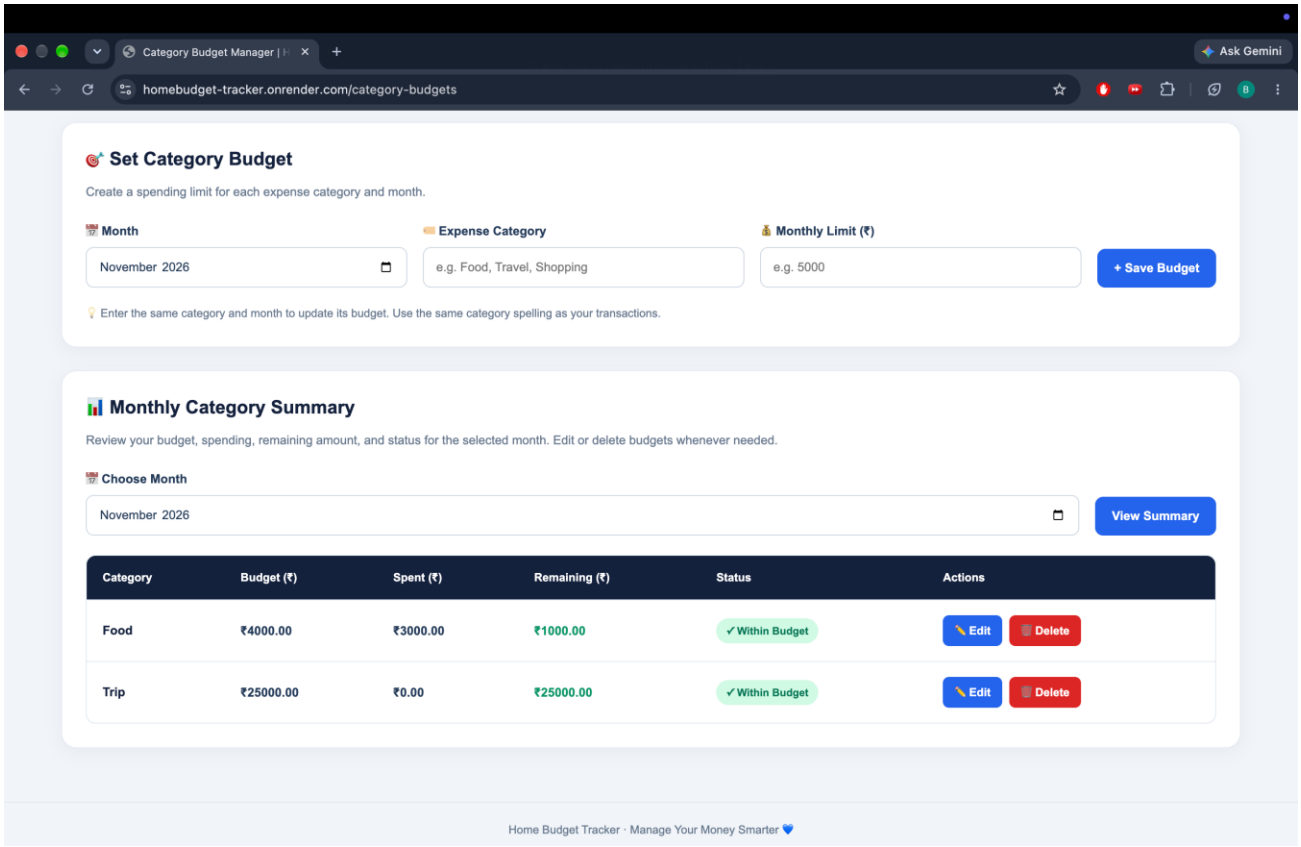


Fig. 13.7: Category Budget Manager

13.8 Savings Goals Page

Shows savings-goal details, target amounts, saved amounts, and progress information.

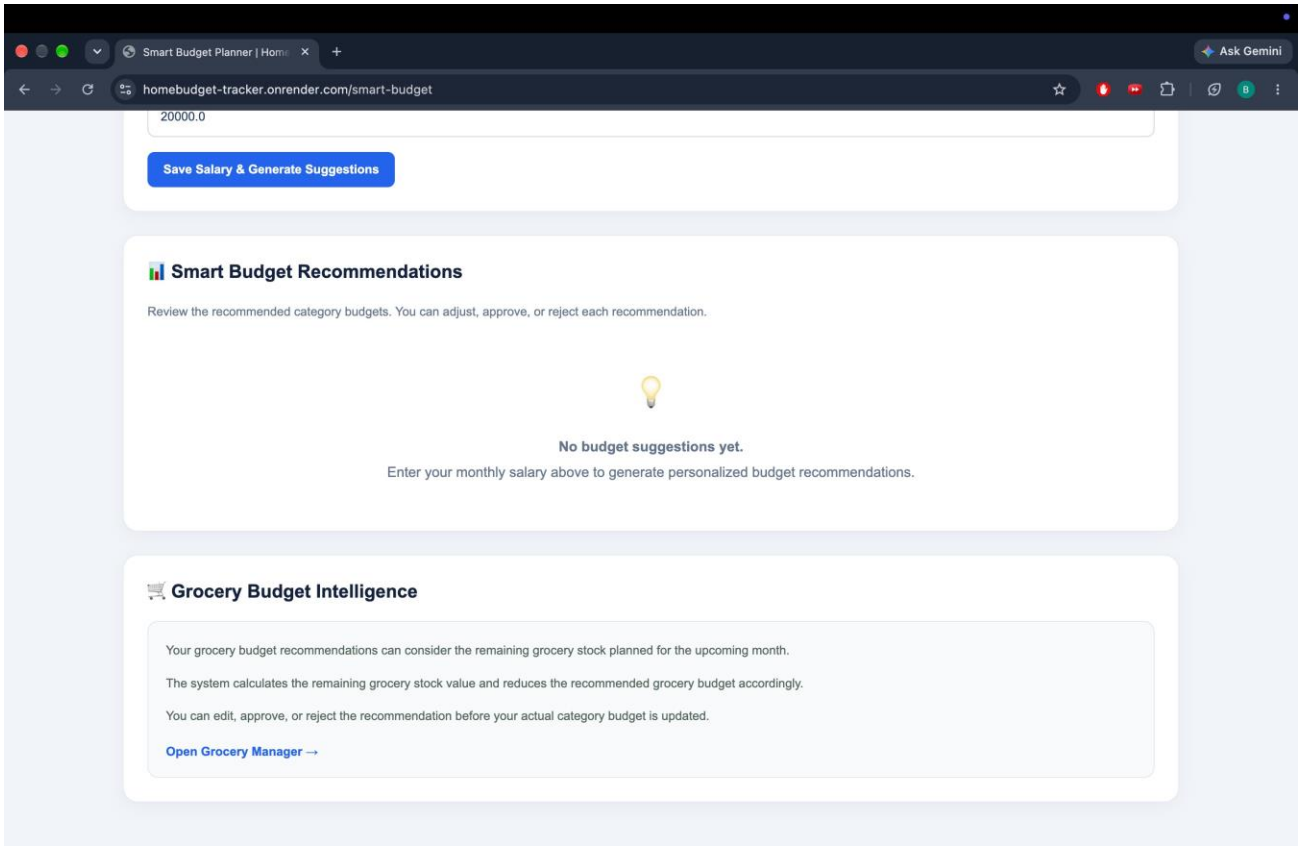


Fig. 13.8: Savings Goals Page

14. TESTING

Software testing is performed to verify that the application modules work according to their requirements. Testing should cover transaction operations, dashboard calculations, database connectivity, budget features, analysis output, and savings-goal operations.

14.1Types of Testing

Functional Testing

Checks that the user can perform supported operations such as adding, editing, viewing, and deleting transactions and maintaining budgets and savings goals.

Database Testing

Checks that records are inserted, retrieved, updated, and deleted correctly and that the application can connect to the configured database.

User Interface Testing

Checks page navigation, forms, buttons, tables and display behavior in a modern web browser.

S. No.	Test Case	Expected Result	Status
1	Open application	Application home or login page should load.	To be verified
2	User login	Valid credentials should allow access, if authentication is enabled.	To be verified
3	Add income transaction	Valid income record should be saved and displayed.	To be verified
4	Add expense transaction	Valid expense record should be saved and displayed.	To be verified
5	Edit transaction	Selected transaction should be updated.	To be verified
6	Delete transaction	Selected transaction should be removed.	To be verified
7	Dashboard totals	Displayed totals should match stored transaction records.	To be verified
8	Monthly budget	Budget information should be saved and displayed correctly.	To be verified
9	Category budget	Category limit and spending comparison should be correct.	To be verified
10	Compare months	Selected months should display the corresponding comparison.	To be verified
11	Analysis charts	Charts should reflect the available transaction data.	To be verified
12	Savings goal	Goal details and saved amount should be stored and displayed.	To be verified
13	Database connectivity	Application should communicate with the configured database.	To be verified

Calculation Testing

Checks that income, expenses, available balance, savings rate, and budget comparisons match the underlying transaction records.

Visualization Testing

Checks that charts and analytical summaries represent the available transaction data.14.2 Test Cases

14.3 Testing Outcome

The test cases listed above provide a checklist for validating the main modules. The final testing outcome must be recorded after testing the deployed or local application with representative data. Any failures should be corrected and the relevant test cases executed again.

15. CONCLUSION

Home Budget Tracker is a web-based application developed to simplify the recording and review of personal and household financial activity. It combines transaction management, dashboard summaries, budget planning, month comparison, category-wise budget management, savings-goal tracking, and data visualization in one application.

The project demonstrates the integration of Python, Flask, HTML, CSS, JavaScript, and a MySQL-compatible database. Pandas and NumPy support data processing, while Plotly provides visual representations of financial information. Deployment through Render makes the application accessible through a web browser.

The project provides practical experience in web application development, database connectivity, financial calculations, testing, integration, and deployment. It can be extended with additional reporting and financial planning features in future versions.

16. FUTURE ENHANCEMENTS

- Add downloadable reports in PDF and Excel formats.
- Introduce recurring transactions and scheduled reminders.
- Provide more detailed financial forecasts and spending insights.
- Add configurable alerts when monthly or category budgets approach their limits.
- Improve savings-goal tracking with milestone notifications.
- Provide database backup and restore facilities.
- Enhance security with stronger authentication and additional session protections.
- Improve mobile responsiveness and accessibility.

- Add optional bank statement import after appropriate validation and security controls.

17. REFERENCES

16. Python Official Documentation – https://docs.python.org/3/
17. Flask Documentation – https://flask.palletsprojects.com/
18. MySQL Documentation – https://dev.mysql.com/doc/
19. MySQL Connector/Python Documentation – https://dev.mysql.com/doc/connector-python/en/
20. Pandas Documentation – https://pandas.pydata.org/docs/
21. NumPy Documentation – https://numpy.org/doc/
22. Plotly Python Documentation – https://plotly.com/python/
23. Render Documentation – https://render.com/docs
24. MDN Web Docs – https://developer.mozilla.org/

LIVE PROJECT LINK

https://homebudget-tracker.onrender.com